

CS121 Data Structures in C

03 *Linear Lists*

Outline

- Introduction
- What is the linear lists?
- The concept of Abstract Data Type (ADT)
- The linear list ADT

Introduction (1)

Example: A grading record of CS121

```
("student-1", 85)
("student-2", 78)
...
("student-N", 90)
```

- Representation: Array “**a**”

The type of items (elements): e.g. **int**, to be simple.

The memory space to hold the record: an array of **ints**.

The size of the array: pre-defined and should be large enough.

```
enum {MAXSIZE = 1024};
```

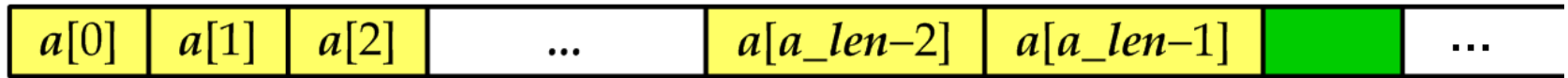
```
int a[MAXSIZE];
```

- The length of the record, or the number of items in the record: initially 0.

```
int a_len = 0;
```

Introduction (2)

- Operation 1: adding an item to the end
supposing the record *a* has *a_len* items

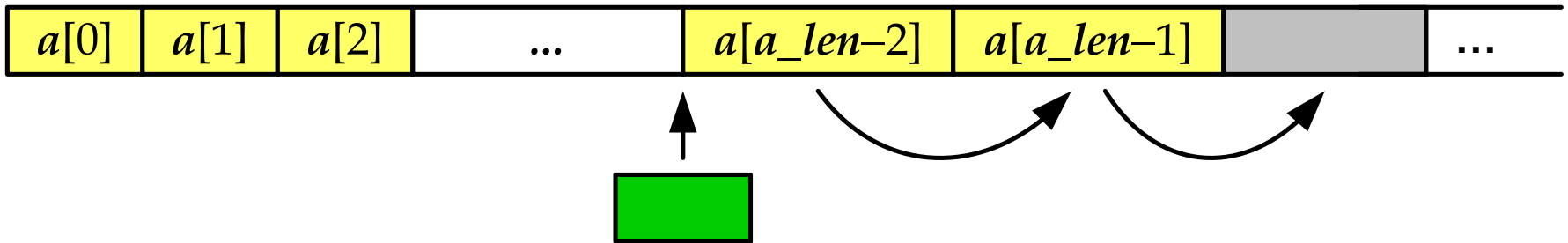


Steps:

- ◆ Check if the array space is large enough.
- ◆ Assign the item to *a*[*a_len*].
 - Array indices are always starting from 0.
- ◆ Increment the length of the list.
 - *a_len* = *a_len*+1.

Introduction (3)

- Operation 2: inserting an item in between
To insert an item at position i



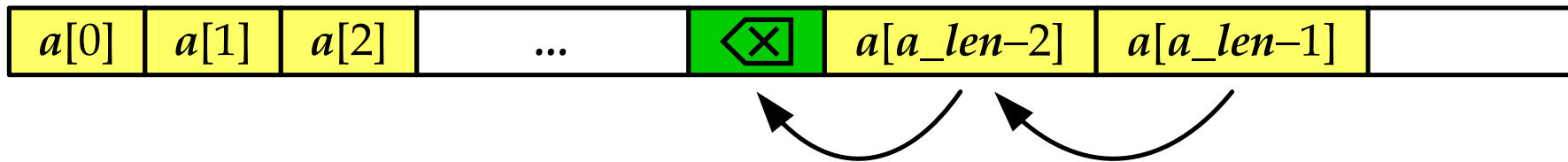
Steps:

- ♦ Check if the array space is large enough.
- ♦ Check if i is in range: $0 \leq i \leq a_len$. (Why a_len ?)
- ♦ Shift the items starting from the insertion point one position to the right. (How?)
- ♦ Assign the item to $a[i]$.
- ♦ Increment the length: $++a_len$.

Introduction (4)

- Operation 3: removing an item

To remove an item at position i



Steps:

- ♦ Check if i is in range: $0 \leq i < a_len$. (What if the collection is empty?)
- ♦ Shift the items next to the deleted point one position to the left. (How?)
- ♦ Decrease the length: $--a_len$.

Linear Lists (1)

- How do we classify a data structure?
 - ♦ The properties.
 - ♦ The operations.
- The array can be the representation of a linear list.
 - ♦ It has finite number of elements (items). It has a length property.
 - ♦ It forms a sequence, where items are arranged one by one. This “one next to another” relation is called linear relation property.

Linear Lists (2)

- Linear: the sequential property
 - ♦ The unique “the first element” which has no predecessor:
Head
 - ♦ The unique “the last element” which has no successor:
Tail
 - ♦ All other elements have the unique predecessor and successor

We can be more concrete to say that in a linear list, each element has its definite position. If the list is modified, some elements' positions may be changed.

Linear List Operations

- Initialize a linear list to an empty list.

We call a list with length 0 the *empty* list.

- Retrieve the length property.
- Retrieve an item at position i .
- Insert an item at position i .
- Remove an item at position i .
- And, given the item's value, locate an item's position.

For example: The List is 12, 15, 98, 33, 76.

Length(List); Retrieve(1); Insert(60, 3); Find(15)

Wrap them up to an ADT

- Abstract Data Type (ADT)

The specification of a set of related operations can be formulated to an *Abstract Data Type*.

- ◆ Mathematical abstractions

- An ADT specifies “what to do”.
- An ADT never mentions the implementation (“how to do”).

- ◆ Encapsulation

- Encapsulate the implementation details
- The ADT forms a layer between the users and the implementation. The users of an ADT need not concern the implementation details, and the underlying implementation can be altered freely.

Linear List ADT (1)

```
typedef struct _T_linear_list linear_list_t;

typedef enum _T_error_t
    {NOERROR, OVERFLOW, SUBSCRIPT}
    error_t;

typedef int elem_t;

/* create and destroy the linear list */
linear_list_t *Create(void);
void Destroy(linear_list_t *L);

/* A set of operations */
void Set_null(linear_list_t *L); /* empty list*/
int Length(linear_list_t *L); /* length of the list */
elem_t Get(linear_list_t *L, int i); /* get the value */
error_t Insert(linear_list_t *L, elem_t x, int i); /* insert */
error_t Remove(linear_list_t *L, int i); /* remove */
int Locate(linear_list_t *L, elem_t x); /* locate the pos.*/
```

Linear List ADT (2)

```
/* create and destroy the linear list */  
linear_list_t *Create(void);  
void Destroy(linear_list_t *L);
```

- Implementation detail to create (initialize) and destroy a list
 - ♦ Use `malloc(sizeof(linear_list_t))` to allocate space.
 - ♦ Use `free(L)` to free space.
 - ♦ Don't forget to initialize certain fields.
- For user: complete encapsulation, using only references (pointers) to access the ADT, without knowing any detail.

Linear List ADT (3)

```
linear_list_t *Create(void)
{
    linear_list_t *L = (linear_list_t*)malloc(sizeof(*L));

    if ( L )
        L->a_len = 0;

    return L;
}

void Destroy(linear_list_t *L)
{
    free(L);
}
```

Linear List ADT (4)

```
/* empty list*/
void Set_null(linear_list_t *L)
{
    L->a_len = 0;
}

/* the length of the list*/
int Length(linear_list_t *L)
{
    return L->a_len;
}

/* get the value at position i*/
elem_t Get(linear_list_t *L, int i)
{
    if ( i < 0 || i >= L->a_len )
        ERROR("Subscript");

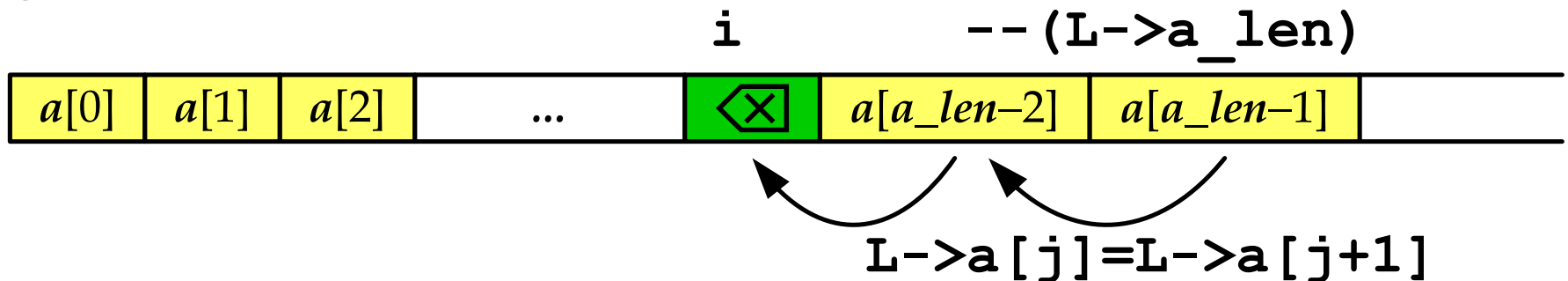
    return L->a[i];
}
```

Linear List ADT (5)

/ remove the item at index i*/*

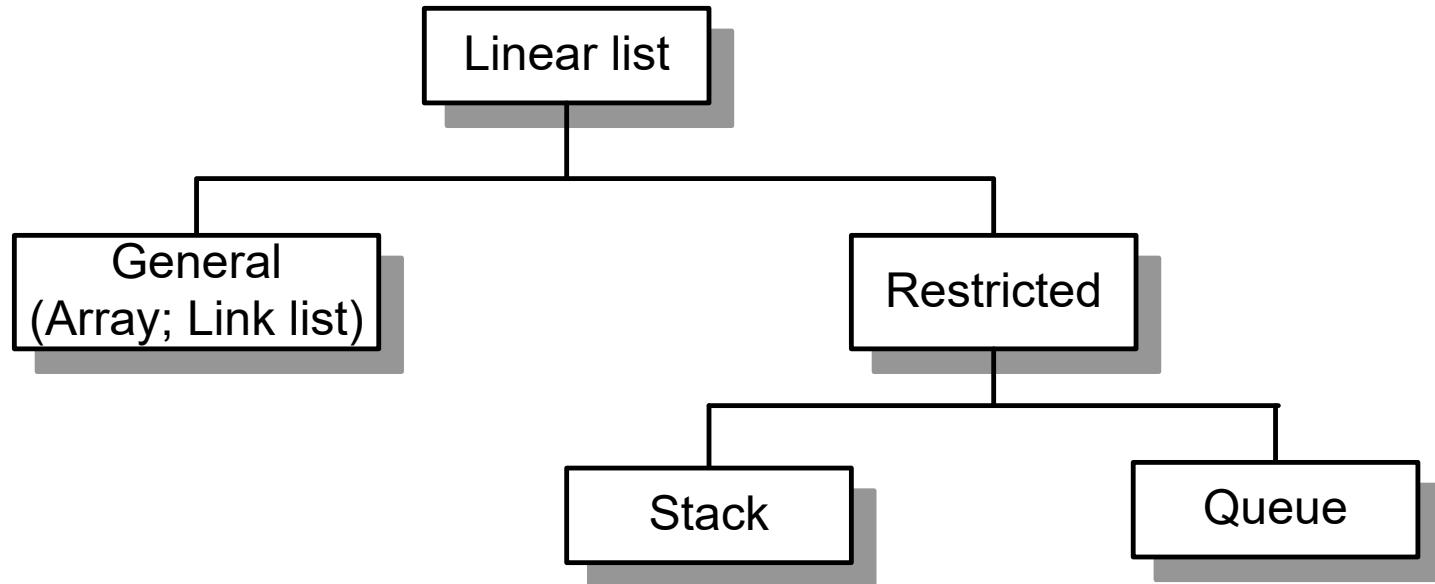
```
error_t Remove(linear_list_t *L, int i)
{
    if ( !(0 <= i && i < L->a_len) )
        return SUBSCRIPT; /* whether 0<= i < L->a_len*/
    else {
        int j;
        --L->a_len; /* decrease the length L->a_len*/
        for ( j = i; j < L->a_len; ++j )
            L->a[j] = L->a[j+1];

        return NOERROR;
    }
}
```



Linear List Categories

- Two categories

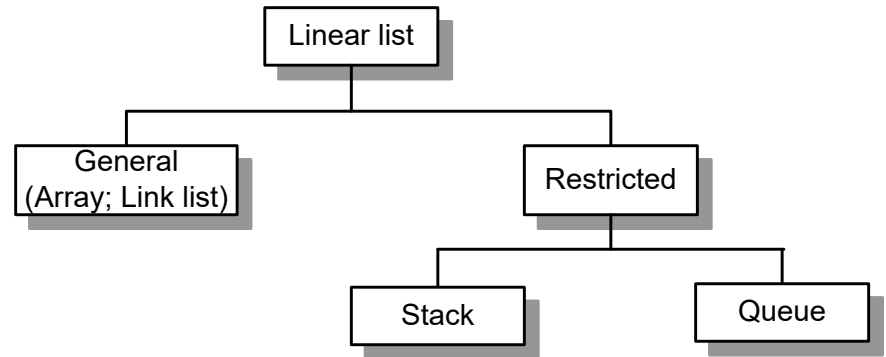


Link list; Stack; Queue

- Link list; Stack; Queue

Linear List Categories

- Two categories



- ♦ General list: element can be inserted or deleted **anywhere** in the list
- ♦ Restricted list: addition and deletion of elements are restricted to the ends of the list.
 - Stack: the **last in-first out** (LIFO) list
 - Queue: the **first in –first out** (FIFO) list

Assignment 02

- Complete the implementation of the following functions in the Linear List ADT
 - ♦ **Insert, Locate and RemoveDup;**
 - ♦ For function **Locate**, it should return -1 to indicate that the desired item is not found.